Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems

YILUN WANG, The Chinese University of Hong Kong, China
GUANGBA YU, The Chinese University of Hong Kong, China
HAIYU HUANG, The Chinese University of Hong Kong, China
YUJIE HUANG, The Chinese University of Hong Kong, China
ZIRUI WANG, The Chinese University of Hong Kong, China
PENGFEI CHEN, Sun Yat-sen University, China
MICHAEL R. LYU, The Chinese University of Hong Kong, China

arXiv:2603.00468v3 [cs.SE] 6 Nov 2026

LLM agents are explored for automating root cause analysis (RCA) in cloud-native systems, creating a need to evaluate both diagnostic correctness and the quality of the supporting investigation. Existing static benchmarks offer repeatable inputs but limited system-facing interaction, while live testbeds expose tools but hinder controlled comparison because incident evidence varies across runs; both paradigms focus primarily on final answers. To address these limitations, we present Cloud-OpsBench, an evaluation infrastructure for interactive and evidence-grounded cloud RCA. It comprises 754 runtime-verified cases across 57 fault types on two microservice workloads spanning application services and Kubernetes platform layers. Each fault is captured as a state snapshot and replayed through standard diagnostic interfaces, with outcome labels and diagnostic evidence graphs that enable matched comparisons and process-level analysis. Evaluating eleven LLMs on Cloud-OpsBench reveals that strong outcome accuracy often masks incomplete investigations: while Joint RCA Accuracy (JRA) reaches up to 0.78 on OnlineBoutique and 0.68 on TrainTicket, models identify the correct root cause without achieving evidence closure (e.g., in 33.1% and 29.4% of evaluated cases for GPT-5). These results demonstrate that final-answer accuracy alone cannot establish whether an agent has traced the failure and collected the evidence needed to confirm its root cause.

CCS Concepts: • **Software and its engineering → Software reliability**.

Additional Key Words and Phrases: Large Language Models, Cloud Computing, Root Cause Analysis, Benchmark, Agentic AI

ACM Reference Format:
Yilun Wang, Guangba Yu, Haiyu Huang, Yujie Huang, Zirui Wang, Pengfei Chen, and Michael R. Lyu. 2026. Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems. 1, 1 (November 2026), 28 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Authors' Contact Information: Yilun Wang, ylwang25@cse.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong SAR, China; Guangba Yu, guangbayu@cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong SAR, China; Haiyu Huang, hyhuang25@cse.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong SAR, China; Yujie Huang, yjhuang@cse.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong SAR, China; Zirui Wang, zrwang@cse.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong SAR, China; Pengfei Chen, chenpf7@mail.sysu.edu.cn, Sun Yat-sen University, Guangzhou City, China; Michael R. Lyu, lyu@cse.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong SAR, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/11-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Cloud-native systems host critical online services, but diagnosing their failures remains difficult because observable symptoms can propagate across application and platform layers [7, 31, 60, 67]. Root-cause analysis is therefore a deployed-software debugging task that spans application code, deployment specifications, runtime configurations, service dependencies, and infrastructure states. LLM-based agents offer a new approach to automating this task by selecting diagnostic tools and targets, interpreting returned observations, refining hypotheses, and consulting operational knowledge [9, 12, 23, 50]. This shift changes the evaluation target from root-cause prediction over supplied telemetry to an interactive investigation whose conclusion must be supported by acquired observations.

Existing RCA benchmarks do not jointly support the conditions needed to evaluate this interactive investigation. Static telemetry benchmarks provide fixed incident artifacts and reproducible inputs, but they do not test whether an agent can select diagnostic tools and targets based on observations returned by the system [41, 45, 69]. Live-environment benchmarks preserve system-facing interaction, but the same injected fault may differ in severity, duration, or affected scope across runs [8, 22, 47]. As a result, two agents evaluated in separate live runs may face different fault conditions, so their performance differences cannot be attributed solely to the agents. Across both paradigms, outcome-oriented metrics evaluate the final diagnosis but cannot determine whether the agent has collected observations that causally justify its conclusion. Consequently, existing evaluations only partially measure interactive evidence acquisition, run-to-run comparability, and diagnostic process quality.

To address these gaps, we present Cloud-OpsBench[1], an evaluation infrastructure for interactive and evidence-grounded cloud RCA. It comprises 754 fault cases across 57 fault types that span application services and Kubernetes platform layers, with OnlineBoutique and TrainTicket serving as representative workloads. Each fault is realized and runtime-verified in a live Kubernetes testbed before its verified incident state and associated temporal telemetry are captured as a state snapshot. During evaluation, snapshot-backed diagnostic interfaces replay the captured responses, allowing different agents to investigate the same realized incident while independently choosing their tools and targets. Each case pairs a root-cause label with a milestone-based evidence graph, enabling evaluation of root-cause correctness, milestone coverage, evidence-order consistency, and investigation efficiency.

We evaluate eleven LLMs on Cloud-OpsBench under a common ReAct scaffold and further examine experience reuse and alternative agent architectures and harnesses. First, diagnostic accuracy varies across models and fault types. Across the eleven models, mean Joint RCA Accuracy (JRA) is 0.59 on OnlineBoutique and 0.49 on TrainTicket, with the highest scores reaching 0.78 and 0.68, respectively. Performance- and code-related faults remain particularly challenging. Second, correct diagnoses can lack complete supporting evidence. For example, GPT-5 produces correct diagnoses with incomplete evidence in 33.1% of evaluated cases on OnlineBoutique and 29.4% on TrainTicket. Trajectory inspection shows that agents can submit the correct RCA result without tracing the failure to the affected backend or performing the checks needed to confirm the root cause. Third, operational experience can improve diagnosis, but its benefits vary across models, systems, and reuse methods. Fourth, with GLM-5.1 held fixed, different agent architectures and harnesses achieve similar diagnostic accuracy with substantially different evidence closure rates and token costs. Higher computational expenditure does not consistently yield more accurate or more complete diagnoses.

In summary, this paper makes the following contributions:

[1] https://github.com/LLM4Ops/Cloud-OpsBench

Table 1. Comparison of Cloud-OpsBench with existing AIOps benchmarks across the three evaluation gaps.

Benchmark	Paradigm	System Interaction *(Gap 1)*	Run-to-Run Consistency *(Gap 2)*	Process Evaluation *(Gap 3)*
Static Telemetry Benchmarks				
LogHub [69]	Static Artifact	✗	✓	✗
RCAEval [41]	Static Artifact	✗	✓	✗
TimeSeriesBench [45]	Static Artifact	✗	✓	✗
Nezha [59]	Static Artifact	✗	✓	✗
OpenRCA [54]	Static Artifact	✗[1]	✓	✗
Dynamic Environment Benchmarks				
AIOpsArena [47]	Live Environment	✗	✗	✗
AIOpsLab [8]	Live Environment	✓	✗	✗
ITBench [22]	Live Environment	✓	✗	✗
SREGym [10]	Live Environment	✓	✗	✗
ORCA-bench [16]	Live Environment	✓	✓[2]	✗
Proposed Benchmark				
Cloud-OpsBench	**State Snapshot**	✓[3]	✓	✓

[1] Code execution over supplied offline artifacts is not counted as system-facing interaction because it does not query an operational interface.
[2] It queries a fixed six-day historical telemetry data.
[3] Cloud-OpsBench replays captured cluster observations through snapshot-backed diagnostic interfaces for supported tool queries.

- **A Runtime-Verified Fault Benchmark.** We construct Cloud-OpsBench with 754 fault cases spanning 57 fault types across application services and Kubernetes platform layers on two microservice workloads. A knowledge-guided pipeline converts fault specifications into executable injections and retains each case only after live runtime verification.
- **Decoupled Fault Realization and Interactive Evaluation.** We separate live fault realization from agent evaluation by capturing the tool-observable evidence of each verified incident as a state snapshot. Snapshot-backed interfaces then replay the same incident under matched observations while allowing agents to choose diagnostic tools and targets independently.
- **Process-Grounded Diagnostic Evaluation.** We represent diagnostic processes using milestone-based evidence graphs that encode completion logic, prerequisite relations, and alternative admissible evidence bundles. We define metrics for milestone coverage, evidence closure, evidence-order consistency, and investigation efficiency, without requiring a single prescribed expert command sequence.
- **Experiments on LLM-based RCA Agents.** We evaluate eleven LLMs on Cloud-OpsBench and reveal a gap between root-cause identification and diagnostic evidence closure: agents can correctly identify the faulty component and fault type without completing the supporting diagnostic evidence chain. These findings highlight that outcome accuracy alone is insufficient to establish the reliability of evidence-grounded diagnosis.

2 Motivation

2.1 Agentic RCA

Cloud-native root-cause analysis is a deployed-software debugging task. In Kubernetes, software behavior is determined jointly by application code, deployment specifications, runtime configurations, service dependencies, and infrastructure states. A failure reported at an entry service may therefore originate several components or system layers away. Diagnosis must connect the

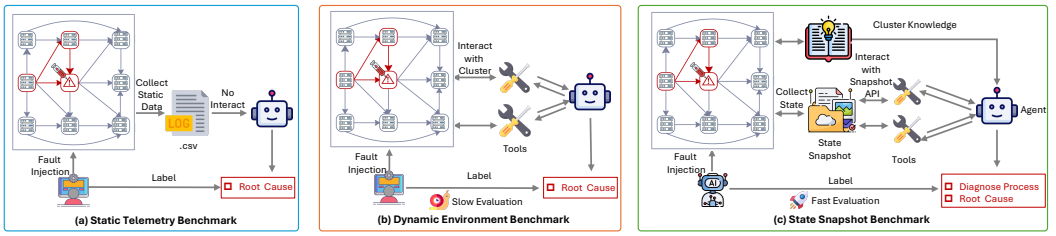

Fig. 1. Benchmarking paradigms for agentic RCA. (a) Static telemetry: no system-facing interaction; outcome-only root-cause labels. (b) Live environments: interactive diagnosis, but still outcome-only evaluation, with run-to-run variability and high cost. (c) State snapshots (Cloud-OpsBench): matched interactive investigation over frozen incident evidence, with both process and outcome scoring.

reported symptom to a concrete faulty component and causal mechanism across these software and operational artifacts. This paper focuses on this task in Kubernetes-based cloud-native systems.

Agentic RCA makes this debugging process interactive. Given a user query describing an observed failure, an agent decides which system objects, logs, configurations, and dependency relations to inspect through diagnostic tools, and revises its hypotheses as observations accumulate [38, 55]. Consequently, two agents may produce the same root-cause label while differing substantially in whether their conclusions are supported by sufficient and causally coherent evidence. A benchmark for agentic RCA must therefore evaluate both the diagnostic outcome and the evidence-acquisition process that supports it.

Agents may also consult operational knowledge, such as API semantics and troubleshooting guidance, when interpreting observations [12, 23, 28]. Such knowledge can be provided as a controlled experimental input, but any accepted diagnosis must still be grounded in case-specific observations returned by tools.

2.2 Evaluation Gaps

Existing benchmarks leave complementary aspects of this investigation unmeasured. Static benchmarks offer reproducible incident artifacts but do not evaluate interactive evidence collection. Live-environment benchmarks support such interaction but expose agents to varying runtime observations, complicating process-level comparison. Across both paradigms, outcome-oriented metrics cannot determine whether the collected evidence justifies the diagnosis. Figure 1 and Table 1 summarize these limitations.

2.2.1 Gap 1: Lack of Interactive Diagnosis in Static Benchmarks. Static RCA benchmarks typically provide pre-collected logs, metrics, or traces as fixed inputs [11, 41, 45, 69]. This design supports reproducible analysis, and preprocessing or code-based filtering can reduce the amount of telemetry presented to a model. However, it does not provide the system-facing interaction used in operational diagnosis, where an agent selects a diagnostic tool, specifies its target, and receives an observation before deciding what to inspect next. Consequently, the benchmark evaluates reasoning over supplied evidence rather than the agent's ability to acquire evidence through an adaptive investigation. Even settings that allow code execution over offline artifacts still operate on a supplied file corpus rather than on system-facing diagnostic interfaces [54].

In a static formulation of the TrainTicket DBReadOnly case, the model receives a fixed corpus of logs and telemetry and searches it offline for evidence such as read-only write rejections. This setting evaluates evidence retrieval and interpretation within a predetermined corpus. It does not evaluate whether, starting only from a query reporting a payment failure, the agent knows to query the dependencies of `ts-payment-service`, select `ts-order-service` from the returned topology,

and request its logs. The missing capability is therefore interactive evidence collection, in which the agent chooses what to query next based on observations returned by previous diagnostic actions.

2.2.2 Gap 2: Variability Across Live Runs. Live-environment benchmarks deploy microservices and inject faults during evaluation, preserving system-facing interaction and realistic runtime behavior [8, 10, 22, 47]. Even when the fault type, target, and injection parameters are held constant, the resulting fault manifestation may vary in severity, duration, and affected scope across runs. The realized impact is jointly determined by the fault injection and the system's transient runtime state. Concurrent workloads, resource availability, scheduling decisions, and in-flight retries can amplify or attenuate the observable failure.

Such variation is valuable for evaluating robustness under realistic operational uncertainty. However, it complicates controlled agent comparison. When two agents are evaluated in separate live runs, both the diagnostic policy and the realized fault manifestation may differ. Observed performance differences therefore cannot be attributed solely to the agents. Repeated trials can estimate average performance and run-to-run variance, but do not guarantee identical tool observations across agents.

In addition, live evaluation incurs substantial lifecycle overhead. Each independent run must establish a clean system state, warm up the workload, activate and verify the fault, allow the agent to complete its investigation, and restore the environment before the next trial. Because reliable estimates require repeated runs for every combination of fault case, agent, and experimental configuration, this lifecycle cost grows multiplicatively with benchmark scale. The resulting evaluation latency and infrastructure demand make large-scale comparison and rapid agent iteration difficult [62].

2.2.3 Gap 3: Outcome-Only Evaluation. Existing RCA benchmarks primarily evaluate whether the predicted component and fault type match the ground truth using metrics such as top-k accuracy, precision, recall, or F1-score [41, 47, 69]. These metrics remain necessary for measuring diagnostic correctness, but they provide little information about how an agent obtains its answer. Two agents receive identical outcome credit when they return the same correct diagnosis, even if one has collected observations that connect the reported symptom to the root cause while the other submits the same diagnosis without collecting key supporting observations. Conversely, an agent that conducts a largely grounded investigation but makes an incorrect final attribution is indistinguishable from one that fails to investigate the system.

This creates a gap in the construct validity of agentic RCA evaluation. The claimed capability includes choosing diagnostic actions, acquiring relevant observations, and establishing a causal explanation, whereas outcome-only metrics measure only the final prediction. Evaluation should therefore complement diagnostic accuracy with process measures over the observable tool-mediated investigation rather than the agent's self-reported reasoning [25, 51]. Such measures should assess the completeness, causal coherence, and efficiency of the investigation.

2.3 Benchmark Requirements

The preceding gaps yield three requirements for evaluating agentic RCA. **R1: System-facing evidence acquisition.** An agent must be able to select tools and targets through a defined set of standard diagnostic interfaces, receive captured system observations, and adapt subsequent actions to the returned evidence. **R2: Matched and reusable incident evidence.** Different agents must investigate the same realized incident under the same available evidence and deterministic query-response mapping without redeploying or reinjecting the fault for each evaluation. **R3: Process evaluation beyond a single reference trajectory.** In addition to outcome correctness, the benchmark must assess the evidential completeness, causal order, and efficiency of the observable

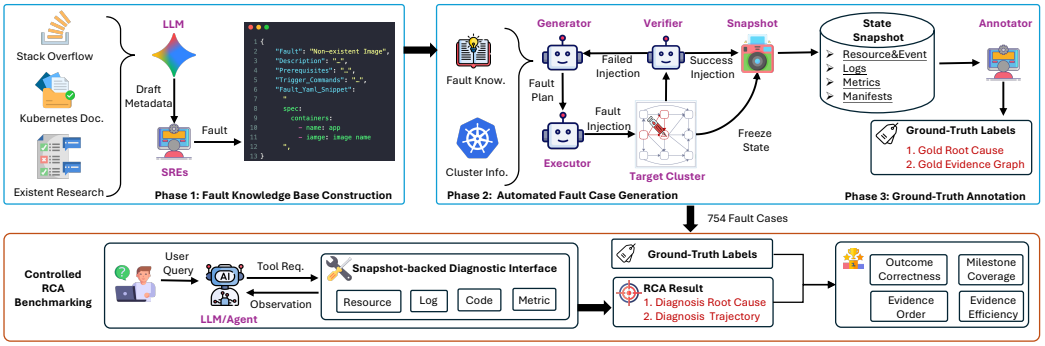

Fig. 2. Overview of Cloud-OpsBench: Phase 1 builds the fault knowledge base; Phase 2 instantiates, verifies, and snapshots each case; Phase 3 annotates outcome and process labels; agents then investigate through snapshot-backed diagnostic interfaces.

investigation without prescribing a canonical command sequence. Evidence-equivalent diagnostic facts and independent investigation orders should receive equal credit. We scope evaluation to interactive investigation over preserved, runtime-verified incident evidence rather than to live remediation or open-ended environment mutation. These requirements guide the design of Cloud-OpsBench, introduced next.

3 Cloud-OpsBench

To satisfy these requirements, we present Cloud-OpsBench, an evaluation infrastructure for interactive and evidence-grounded cloud RCA comprising 754 fault cases across 57 fault types that span application services and Kubernetes platform layers. As illustrated in Fig. 2, Cloud-OpsBench realizes each fault in a live Kubernetes testbed through a knowledge-grounded construction pipeline (§ 3.2–3.3). Snapshot-backed diagnostic interfaces satisfy R1 by supporting adaptive investigation through a defined diagnostic toolset (§ 3.1). They satisfy R2 by replaying the same available incident evidence and query-response mapping without per-agent fault realization. Each case carries a milestone-based evidence graph that satisfies R3 by evaluating established diagnostic facts and their prerequisite relations without requiring agents to follow a single reference command sequence (§ 3.4).

3.1 Design Philosophy and Task Formulation

To bridge the tension between ecological validity and experimental control, we introduce the **State Snapshot** paradigm shown in Fig. 1(c). Instead of evaluating agents directly against a live cluster, Cloud-OpsBench first realizes each fault in a real Kubernetes environment and captures its tool-observable context after the Verifier confirms the task-defining manifestation. Each snapshot combines resource, configuration, topology, and component states collected after verification with timestamped events, logs, and metrics from the case-specific incident interval between pre-injection baseline confirmation and manifestation verification. It therefore preserves both the verified incident state and its associated temporal telemetry without attempting to reproduce the complete system history.

During evaluation, agents interact with a snapshot-backed diagnostic interface that replays captured responses to supported tool queries, such as calling `GetResources` to inspect Pods. This design does not eliminate noise from the original incident evidence. Rather, it eliminates evaluation-time environmental nondeterminism: the snapshot preserves evidence produced by a real fault realization while fixing the observation space for agent comparisons.

Within this controlled replay environment, we formalize Agentic RCA as a trajectory-based decision process. For each evaluation case j, the outcome ground truth is $\mathcal{R}_j^* = \langle C_j^*, F_j^* \rangle$, where C_j^* denotes the faulty component and F_j^* denotes the fault type. Its process-level ground truth is represented by a diagnostic evidence graph $\mathcal{D}_j^*$, which specifies the intermediate diagnostic states required to justify $\mathcal{R}_j^*$ and is constructed in § 3.4. An evaluation episode is modeled as

$$\Phi : \langle U_j, \mathcal{S}_j \rangle \rightarrow \langle \mathcal{T}_j, \hat{\mathcal{R}}_j \rangle. \tag{1}$$

The agent receives a user query U_j describing the observed failure, diagnostic tool specifications, and the allowed component and fault-type labels. Case-specific system observations are obtained through the diagnostic interface backed by the frozen snapshot $\mathcal{S}_j$, while the underlying snapshot repository is not directly exposed to the agent. At each step, the agent issues a concrete tool use t_i, including the tool name and arguments, and the interface returns an observation o_i replayed from $\mathcal{S}_j$. The evaluation harness records the resulting trajectory $\mathcal{T}_j = [(t_1, o_1), \ldots, (t_n, o_n)]$. At termination, the agent returns only the predicted diagnosis $\hat{\mathcal{R}}_j = \langle \hat{C}_j, \hat{F}_j \rangle$. We evaluate $\hat{\mathcal{R}}_j$ against $\mathcal{R}_j^*$ for outcome accuracy and $\mathcal{T}_j$ against $\mathcal{D}_j^*$ for process quality.

3.2 Phase 1: Fault Knowledge Base Construction

Cloud-OpsBench builds a fault taxonomy that spans both upper-layer microservices and the underlying infrastructure, synthesized from three complementary sources: (1) official Kubernetes documentation [24] for specification-based misconfigurations (e.g., incorrect service selectors); (2) developer communities [26, 46] for empirical operational faults; and (3) academic literature [4, 8, 22] for canonical failure modes.

Since raw sources combine unstructured prose with code snippets, we use Gemini 3 Pro as a structured extraction tool via few-shot prompting. For each fault type, the extraction process produces a reusable fault-knowledge entry comprising semantic metadata and a parameterized reproduction template. The metadata records the fault name, causal mechanism, expected local failure state, and possible upper-layer manifestations. The template specifies abstract prerequisites, the fault-inducing mutation or perturbation, and its activation logic using typed variables rather than workload-specific resource identifiers. For example, a service selector mismatch template requires the selector of a Service to differ from the labels of its intended backend Pods while leaving the namespace, Service, Pods, label keys, and label values unbound.

Candidate fault-knowledge entries undergo two pre-instantiation checks. First, an LLM-based consistency check identifies incomplete, ill-formed, or technically inconsistent specifications and refines their structured descriptions. Second, reviewers with cloud-native operations experience assess the causal logic of each fault and the technical feasibility of its reproduction template. Runtime validation is then performed on every case-specific instantiation in Phase 2. A fault type is retained in the released taxonomy only after at least one instantiated case passes this validation. This separation prevents an extracted template from being treated as executable or valid before it is instantiated and tested in a real cluster. Ultimately, Cloud-OpsBench contains 57 specific root-cause types across 8 categories spanning the Kubernetes stack.

We further tag each fault category with a diagnostic difficulty label (Easy/Medium/Hard) that reflects how hard it is to localize the faulty component and confirm its mechanism from the reported symptom (Tab. 2). The label is assigned at construction time based on how directly the fault is exposed by observable symptoms and how much cross-component or cross-layer investigation is typically needed. It is assigned at the category level and shared by all cases within that category.

Table 2. Fault Taxonomy in Cloud-OpsBench. Easy/Medium/Hard correspond to the diagnostic reasoning complexity.

Fault Category *(Mechanism Description)*	**Specific Fault Types (Full List)**	**Difficulty**	**# Cases**
Admission Control Requests rejected by API server due to quota or permission violations.	NamespaceCPUQuotaExceeded, NamespaceMemoryQuotaExceeded, NamespacePodQuotaExceeded, NamespaceServiceQuotaExceeded, NamespaceStorageQuotaExceeded, MissingServiceAccount	Medium	58
Scheduling Pods remain Pending due to unsatisfied scheduling constraints or volume-binding requirements.	NodeCordonMismatch, NodeAffinityMismatch, NodeSelectorMismatch, PodAntiAffinityConflict, TaintTolerationMismatch, CPUCapacityMismatch, MemoryCapacityMismatch, PVBindingOccupied, PVCSelectorMismatch, PVCStorageClassMismatch, PVCCapacityMismatch, PVCAccessModeMismatch	Easy	164
Startup Containers fail to initialize or start.	VolumeMountPermissionDenied, MissingSecretBinding, IncorrectImageReference, ImageRegistryDNSFailure, MissingImagePullSecret	Easy	86
Runtime Runtime failures involving application execution, health checks, backend access, or replica availability.	OOMKilled, LivenessProbeIncorrectProtocol, LivenessProbeIncorrectPort, LivenessProbeIncorrectTiming, ReadinessProbeIncorrectProtocol, ReadinessProbeIncorrectPort, ServiceSidecarPortConflict, MysqlInvalidCredentials, MysqlInvalidPort, DBReadOnly, DBConnectionExhaustion, DeploymentZeroReplicas	Easy	141
Service Routing Traffic routing failures between internal components.	ServiceSelectorMismatch, ServicePortMappingMismatch, ServiceProtocolMismatch, ServiceEnvVarAddressMismatch,GatewayMisroute,ServiceDNSResolutionFailure	Medium	91
Performance Performance degradation due to saturation or network drops.	PodCPUOverload, PodNetworkDelay,NodeNetworkDelay, NodeNetworkPacketLoss	Hard	76
Infrastructure Outages in underlying cluster control plane or components.	ContainerdUnavailable, KubeletUnavailable, KubeProxyUnavailable, KubeSchedulerUnavailable	Medium	40
Application Code Defect Application-level code defects in key business logic.	CodeMissingParameter, CodeBusyLoop, CodeExcessiveFileReads, CodeWrongReturn, CodeExcessiveFileWrites, CodeWrongArgumentOrder, CodeArtificialDelay, CodeMemoryLeak	Hard	98
Total	**57 distinct fault types**	-	**754**

3.3 Phase 2: Automated Fault Case Generation

Traditional fault injection relies heavily on manual orchestration, which is labor-intensive and difficult to scale. As illustrated in Fig. 2, Phase 2 uses a three-agent closed loop to instantiate the fault-knowledge entries from Phase 1 into executable fault plans and verified benchmark cases.

Testbed. To instantiate the fault cases, we deploy a four-node Kubernetes (v1.31) testbed on Huawei Cloud ECS, using `c6.2xlarge.2` instances with 8 vCPUs and 16 GiB of memory per node. As the foundation of cloud-native infrastructure, Kubernetes offers not only a complex distributed control plane for fault injection but also a standardized corpus of documentation. This allows us to rigorously test knowledge-grounded reasoning. The observability plane integrates Prometheus [42] and Istio [21] for metrics, alongside native APIs for object states and logs. As target workloads, we deploy two widely adopted microservice benchmarks, **OnlineBoutique** (11 microservices) [17] and **TrainTicket** (41 microservices with database-backed services) [68], and generate user traffic with Locust [36]. We use ChaosBlade [6] to inject CPU load, network delay, and packet loss, and `kubectl` commands to introduce configuration faults. Together, these mechanisms realize faults across application services and Kubernetes platform layers. For code-defect faults, we mutate service implementations with representative backend bug patterns, such as missing parameter values, wrong return values, or incorrect exception handling following [58]. Each mutated implementation is packaged into a service image and deployed to the testbed.

3.3.1 Fault Injection Loop. For each candidate case j, the *Generator Agent*, implemented with Gemini 3 Pro, produces a case-specific fault plan $\mathcal{B}_j$ from a fault-knowledge entry and the target workload. Each plan specifies the required pre-injection state, resource selectors and operational parameters, ordered injection operations, and runtime validation assertions. The validation assertions cover mechanism activation, the expected local failure state, and any applicable upper-layer manifestation. Before each injection attempt, the *Executor Agent* restores the testbed to a clean

Table 3. Diagnostic tools exposed by Cloud-OpsBench.

Tool Name (Arguments)	**Returned Evidence**
`T1.GetResources`(type, namespace, name, flags)	Resource lists or selected objects with status, metadata, and extended attributes.
`T2.DescribeResource`(type, name, namespace)	Runtime details of a Kubernetes object, including state, conditions, and events.
`T3.GetAppYAML`(app_name)	Deployment configuration YAML for the target application service.
`T4.GetServiceDependencies`(service_name)	Service dependency graph rooted at the target service.
`T5.GetRecentLogs`(service_name, namespace)	Recent service logs for runtime error inspection.
`T6.CheckServiceConnectivity`(namespace, svc_name, port)	TCP reachability result for the target service endpoint.
`T7.GetClusterConfiguration`()	Cluster-wide node information, including resources, labels, taints, and status.
`T8.GetAlerts`()	Metric anomaly alerts generated by threshold-based detectors.
`T9.GetErrorLogs`(service_name, namespace)	Summary of log lines matching error, exception, or failure patterns.
`T10.CheckNodeServiceStatus`(node_name, component)	Liveness status of node-level or control-plane components.
`T11.ListCodeFiles`(app_name)	Available source files for an application service with brief file descriptions.
`T12.GetSourceCode`(app_name, file_path)	Source code content for a selected application file.

deployment, resolves the selectors against the restored cluster state, establishes the required prerequisites, and confirms the pre-injection health baseline, including a successful end-to-end request for service-facing cases, before applying the current fault plan.

The *Verifier Agent* evaluates the validation assertions against the resulting cluster state using object states, events, logs, metrics, and connectivity results. It accepts an execution only when the fault-inducing condition is active at the target component, the expected local failure state is present, and any specified upper-layer manifestation is observed. After successful verification, the resolved target and validated fault condition are recorded in the verified fault plan. This verification is necessary because Kubernetes recovery mechanisms may mask an injected fault. For example, tainting one node may fail to induce a scheduling failure because a newly created Pod can still be placed on another eligible node. If validation fails, the Verifier compares the observed runtime state with the intended fault mechanism and returns structured feedback to the Generator, such as missing prerequisites, insufficient perturbation intensity, or an unintended recovery path. The Generator may then revise resource selectors or operational parameters while preserving the fault mechanism and validation criteria defined by the fault-knowledge entry. The clean deployment, injection, and verification procedure then repeats. Cases that still fail validation after at most three correction attempts are rejected rather than included in the benchmark.

A case is admitted to the snapshot-construction stage only after all runtime validation assertions are satisfied. After the Verifier confirms the observable manifestation used to define the task, we formulate U_j from that manifestation while excluding the faulty component, fault mechanism, and other construction-only information. The verified fault plan is retained as construction provenance but is neither exposed to the evaluated agent nor accepted as diagnostic evidence. Across the construction process, 800 candidate fault plans were generated, and 46 were rejected because they failed to manifest stable or diagnosable symptoms.

3.3.2 State Snapshots. After a fault case passes runtime verification, Cloud-OpsBench materializes its tool-observable context as a state snapshot (§ 3.1). The snapshot captures system state after verification and retains timestamped events, logs, and metrics from the corresponding case-specific incident interval. The snapshot builder records the baseline-confirmation and manifestation-verification timestamps and applies these boundaries when retrieving temporal evidence. To construct the replay interface, we pre-execute the diagnostic toolset (T1–T12, Table 3) against this verified incident context and store the resulting tool responses in a JSON repository. This repository serves as the persistence layer for deterministic replay. The snapshot-backed replay is therefore not a hand-written simulator. During evaluation, it returns the corresponding precomputed response. Before

lookup, the interface validates each request against the supported tool schema and canonicalizes its critical arguments so that equivalent requests map to the same repository entry. Replay coverage is limited to the tool queries supported by the preserved snapshot. Queries outside this coverage receive tool-specific error or empty-result responses.

The snapshot builder covers four evidence channels. First, resource state and events are captured by executing retrieval and description tools (T1, T2) over Kubernetes objects in the target namespace. Second, application topology and telemetry are captured by retrieving static configurations (T3), dependency graphs (T4), logs (T5, T9), and reachability results (T6). Third, cluster-level context is captured by collecting node and component health information (T7, T10) and converting metrics into structured alerts (T8). Fourth, source-code evidence is captured by listing available code files and retrieving source code (T11, T12). These four channels cover the standard evidence sources used in Kubernetes troubleshooting.

Although replay responses remain fixed during evaluation, timestamped logs, Kubernetes events, and alerts comparing normal and abnormal operation preserve temporal information about the incident. For faults such as memory leaks and database connection exhaustion, agents can combine this information with resource states, configurations, and, where available, source code to identify the underlying mechanism.

This design provides faithful replay with an explicit coverage boundary. For supported queries, the interface returns outputs captured from a verified incident in a real cluster rather than responses generated by a manually encoded model of Kubernetes behavior. Coverage is finite and measurable because the construction sweep enumerates the supported tools over case-valid arguments derived from the objects, namespaces, service endpoints, nodes, and source files available in each verified case. Snapshots for OnlineBoutique and TrainTicket contain an average of 599 and 1,708 precomputed responses per case and occupy 3.46 and 8.75 MiB, respectively. A case is retained only when subsequent annotation validation confirms that all required diagnostic facts are observable through the snapshot-backed interface. The finite replay repository bounds the observable evidence, but agents must independently choose tools and arguments as the investigation unfolds rather than follow a prescribed query list or command sequence. Through this construction, state snapshots address Gap 1 and Gap 2: agents investigate via system-facing diagnostic tools rather than offline telemetry dumps, while different agents face the same runtime-verified incident evidence and deterministic query–response mapping.

3.4 Phase 3: Ground-Truth Annotation

Given the verified fault cases and snapshots produced in Phase 2, we annotate both the root cause and the diagnostic evidence needed to support it. For each verified case j, Cloud-OpsBench defines outcome and process labels,

$$\mathcal{G}_j^* = \left\langle \mathcal{R}_j^*, \mathcal{D}_j^* \right\rangle .$$

The outcome label $\mathcal{R}_j^* = \langle C_j^*, F_j^* \rangle$ identifies the faulty component and fault type and is derived from the runtime-verified fault plan $\mathcal{B}_j$. The process label $\mathcal{D}_j^*$ specifies the diagnostic evidence process associated with $\mathcal{R}_j^*$.

3.4.1 Process Representation. For each case j, the process ground truth is represented as

$$\mathcal{D}_j^* = \left\langle \mathcal{M}_j^*, \phi_j^*, \prec_j^*, \mathcal{Q}_j^* \right\rangle ,$$

where $\mathcal{M}_j^*$ denotes the set of diagnostic milestones, ϕ_j^* specifies the completion logic over these milestones, $\prec_j^*$ defines prerequisite relations among milestones, and $\mathcal{Q}_j^*$ specifies the admissible evidence bundles for establishing each milestone.

Table 4. Representative mechanism-relevant objects and states by fault category.

Fault Category	Relevant Objects / States
Admission Control	Namespace quotas, resource requests, service-accounts, resource creation states
Scheduling	Pod scheduling states/events, workload–node constraints, PV/PVC requirements and binding states
Startup	Container startup states, failure logs, image-pull configuration, Secret references, volume/mount configuration
Runtime	Workload replica, container lifecycle states, resource limits, probes, application/database configuration, runtime error logs
Service Routing	Service configuration, gateway routes, application connection addresses, DNS configuration, backend Pods, endpoint states, service connectivity
Performance	Resource/performance alerts, Pod/node health and placement, service dependencies
Infrastructure	Node health, Kubernetes component states, container runtime states, Pod states, service connectivity
Application Code Defect	Source code, container health, runtime error logs, performance symptoms, service dependencies

Each milestone $m \in \mathcal{M}_j^*$ represents a diagnostic fact to be established during the investigation. The case-specific completion formula ϕ_j^* combines milestones using conjunction and disjunction. Conjunctive terms specify milestones or branches that must be completed jointly, whereas disjunctive terms specify alternative completion paths. The diagnostic evidence process is complete when ϕ_j^* evaluates to true over the set of established milestones. The directed acyclic relation $\prec_j^*$ records immediate prerequisite relations among milestones. For $m_a \prec_j^* m_b$, milestone m_a is a prerequisite of m_b. Its transitive closure defines the corresponding partial order, while milestones not connected by a dependency path remain unordered.

For example, consider $\phi_j^* = M_1 \wedge [M_2 \vee (M_3 \wedge M_4)]$, with prerequisite relations $M_1 \prec_j^* M_2$, $M_1 \prec_j^* M_3$, and $M_3 \prec_j^* M_4$. Here, ϕ_j^* defines the milestone combinations that satisfy the completion condition, while $\prec_j^*$ specifies the prerequisite structure among these milestones.

For each milestone m, $\mathcal{Q}_j^*(m)$ contains the admissible evidence bundles that can establish it. Each atomic evidence clause q specifies a diagnostic tool, admissible argument configurations, and, where applicable, a predicate over the normalized response. An evidence bundle is satisfied when all of its clauses have been observed, and a milestone is established when at least one admissible bundle is satisfied.

3.4.2 Annotation Construction. Our process annotations target a complete evidence chain from the reported symptom to the underlying fault mechanism. The milestones capture relevant symptoms, component relationships, and faulty configuration or runtime evidence that explain how the fault produced the observed impact. This evidence chain provides a basis for diagnostic reporting and subsequent remediation. Annotations are constructed in three stages.

Step 1: Constrain the candidate evidence space. Each verified case is associated with a known fault mechanism, which determines the system objects and runtime states relevant to its diagnosis. These case-relevant states are preserved in the frozen snapshot and together form a finite candidate evidence space from which diagnostic milestones and their supporting evidence are identified. Table 4 summarizes the principal object and state families associated with the eight fault categories.

Step 2: Identify and structure diagnostic milestones. Annotators identify diagnostic facts from the candidate evidence space and encode them as milestones in $\mathcal{M}_j^*$. Facts that can be established independently are represented as separate milestones. The milestones are then organized by the completion formula ϕ_j^*. Conjunctive branches encode evidence that must be established jointly,

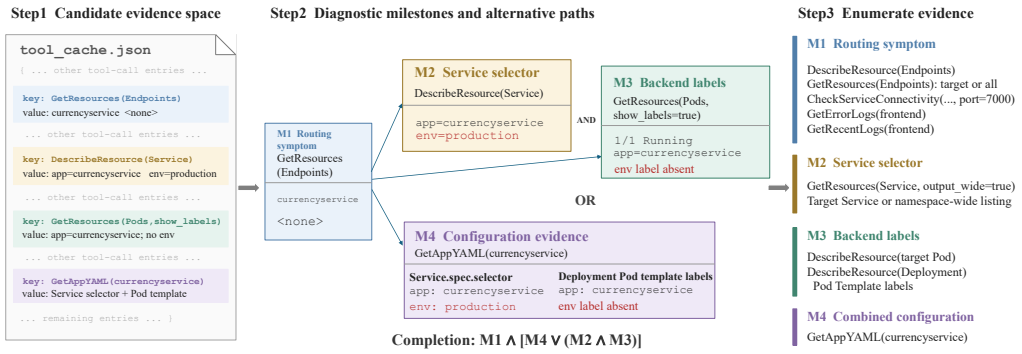

Fig. 3. Annotation construction for OnlineBoutique case `service/50`: identifying relevant observations in the frozen snapshot, structuring diagnostic milestones and alternative completion paths, and enumerating admissible evidence.

while disjunctive branches encode alternative diagnostic paths. Prerequisite relations among milestones are recorded in $\prec_j^*$, producing the partial-order structure used for process evaluation.

Step 3: Enumerate admissible evidence for each milestone. For each milestone, because each snapshot repository is finite and exhaustively indexed, we systematically scan all captured responses across T1–T12 to identify every tool output that exposes the target diagnostic fact. Each evidence clause specifies a diagnostic tool, one or more valid argument configurations, and, when necessary, the diagnostic observation that must be present in the returned result. A milestone may therefore be established through different tool choices, query scopes, or argument configurations that expose the same diagnostic evidence. When a single observation is sufficient, it forms a singleton evidence bundle; when multiple complementary observations are required, they are grouped into a joint bundle. All admissible evidence bundles for milestone m are recorded in $Q_j^*(m)$.

Figure 3 illustrates the three annotation stages using OnlineBoutique case `service/50`, a Service selector mismatch affecting `currencyservice`. The known fault mechanism directs annotation to the Service, its endpoints, and the backend workload preserved in the snapshot. The relevant observations establish a routing symptom, illustrated by the empty endpoint set (M_1), a Service selector requiring both `app=currencyservice` and `env=production` (M_2), and backend Pod labels lacking `env=production` (M_3). The application YAML provides an alternative source of combined configuration evidence (M_4), exposing both the Service selector and the Deployment's Pod-template labels. The completion formula is therefore $M_1 \wedge [M_4 \vee (M_2 \wedge M_3)]$. The annotated dependencies place M_1 before the configuration investigation, while leaving M_2 and M_3 unordered.

The admissible evidence bundles connect these milestones to concrete tool calls and their returned observations. For example, M_2 can be established by describing the Service or retrieving its wide resource listing. For M_3, the backend labels can be obtained through a Pod listing with `show_labels=true`, a description of the target Pod, or the Pod-template information in the Deployment description. These alternatives establish the same diagnostic facts through different queries.

Annotation Validation. Each annotation is first automatically checked against its frozen snapshot. The validation verifies the structural consistency of the completion formula and prerequisite relations, and replays the admissible evidence bundles to ensure that their tool calls and returned observations are valid for the corresponding case. Two auditors with practical experience in Kubernetes operations and cloud fault diagnosis independently review each annotation for diagnostic

sufficiency, milestone granularity, completion-logic validity, and prerequisite correctness, specifically verifying whether alternative tool channels (e.g., startup logs versus object descriptions or YAML manifests) could independently establish the same milestone. They inspect $\mathcal{R}_j^*$, $\mathcal{D}_j^*$, and the corresponding frozen snapshot. Disagreements and proposed revisions are adjudicated by a third senior practitioner with expertise in cloud-native operations.

Following independent audit and adjudication, 9.4% of the annotations were revised and automatically revalidated. The retained annotations contain, on average, 3.0 milestones and 3.1 admissible evidence bundles per milestone.

4 Evaluation

In this section, we evaluate LLM agents on Cloud-OpsBench to answer the following RQs:

- **RQ1 (Outcome Effectiveness):** How accurately do agents identify the faulty component and fault type?
- **RQ2 (Evidence Process):** Do agents collect the required evidence and follow the annotated diagnostic prerequisites?
- **RQ3 (Experience Reuse):** Can RCA agents benefit from historical diagnostic experience?
- **RQ4 (Agent Architectures and Harnesses):** How do different agent implementations affect diagnostic accuracy, evidence completeness, and computational cost?

Additionally, we analyze recurring failure modes in incorrect diagnoses and evidence gaps in correct diagnoses through trajectory analysis.

4.1 Experimental Setup

Setup and Models. We implement a baseline agent framework based on the standard ReAct [56] paradigm, utilizing Pydantic [43] for strict tool parameter validation. This custom architecture natively ensures full-trace observability of the agent's reasoning trajectories and tool invocations. For the ReAct-based experiments in RQ1–RQ3, we use a maximum of 20 interaction steps and set the temperature to 0 to reduce sampling variability. The agent configurations and diagnostic tool-call budget used in RQ4 are specified separately. We evaluate eleven models covering both **proprietary LLMs** and **open-weight LLMs**. The proprietary LLMs include GPT-5, Claude-Sonnet-4, Gemini-2.5-Pro, Gemini-2.5-Flash, and Qwen3.5-Plus. The open-weight LLMs include GLM-5.1, DeepSeek-V4-Flash, Qwen3-235B-A22B, Qwen3.5-27B, Qwen3-14B, and Qwen3-8B, covering high-capacity and compact variants[64].

Diagnostic-difficulty strata. We report selected results stratified by the construction-time diagnostic difficulty labels defined in § 3.2 (Tab. 2). Each evaluation case inherits its category label; we do not reassign difficulty from agent scores. We report each model's results separately within each system and difficulty group.

4.1.1 Evaluation Metrics. We evaluate agents along three complementary dimensions: outcome correctness, process grounding, and diagnostic efficiency.

(1) Outcome Correctness. Our primary metric is **Joint RCA Accuracy** (JRA), the fraction of episodes for which $\hat{\mathcal{R}}_j = \mathcal{R}_j^*$, equivalently, $\hat{C}_j = C_j^*$ and $\hat{F}_j = F_j^*$. We also report **Component Accuracy** (CA) and **Fault-Type Accuracy** (FA) to distinguish localization errors from fault-type classification errors. An episode with a missing or invalid field is counted as incorrect for the corresponding accuracy metric.

(2) Process Grounding. We evaluate each trajectory $\mathcal{T}_j$ against its diagnostic evidence graph $\mathcal{D}_j^*$ (§ 3.4), following process supervision [35] and milestone-based agent evaluation [37, 39]. A

milestone is established when the tool observations satisfy at least one of its admissible evidence bundles. Let $\mathcal{M}_j^{\text{est}}$ denote the set of established milestones.

Milestone Coverage (MC) measures partial completion of the annotated completion formula ϕ_j^*. An established milestone receives a score of 1, and an unestablished milestone receives 0. We evaluate the formula from its milestones upward: each AND node takes the mean score of its child requirements, and each OR node takes the highest score among its alternative branches. Let $s(\phi, S)$ denote the resulting score for formula ϕ when the established milestone set is S. We define $\text{MC}_j = s(\phi_j^*, \mathcal{M}_j^{\text{est}})$. MC therefore credits alternative diagnostic paths without requiring every branch to be completed, and additional exploratory calls do not reduce its value.

Evidence-Order Consistency (EOC) measures milestone coverage under the annotated diagnostic prerequisites. We scan the trajectory chronologically. A milestone receives order credit when its evidence bundle is completed or refreshed after, or in the same tool call as, all its prerequisites receive order credit. Evidence collected too early can receive credit through a later observation that completes or refreshes its bundle after the prerequisites are established. Let $\mathcal{M}_j^{\text{ord}}$ contain the milestones satisfying this rule. We compute $\text{EOC}_j = s(\phi_j^*, \mathcal{M}_j^{\text{ord}})$ using the same scoring function as MC. Independent milestones may be investigated in either order.

Evidence Closure Rate (ECR) measures whether an investigation completes the annotated evidence requirements. For each case, ECR equals 1 when the completion formula is fully satisfied, equivalently when $\text{MC}_j = 1$, and 0 otherwise. Completing one alternative path is sufficient; evidence acquisition order is evaluated separately by EOC. We report MC and EOC as means over evaluated cases, and ECR as the proportion of cases satisfying the closure condition.

(3) Diagnostic Efficiency. Evidence Efficiency (EE) measures the proportion of diagnostic tool calls that establish or refresh milestone evidence. A call receives credit when its observation satisfies an admissible singleton evidence bundle or completes or refreshes a joint evidence bundle. Each call is counted at most once, even if it matches multiple milestones. Repeated calls may receive EE credit when they satisfy the same evidence conditions; acquisition order is evaluated separately by EOC.

Redundant Action Rate (RAR) measures the proportion of diagnostic tool calls that repeat a previously issued tool-and-argument pair. It complements EE by identifying repeated queries, including those that retrieve milestone-relevant evidence.

We also report **Steps** to measure investigation length. In the main ReAct experiments, this count includes diagnostic tool calls and the final diagnosis submission. In RQ4, Steps counts diagnostic tool calls only, excluding planning and control calls. To compare the cost-efficiency of different agent architectures and harnesses, we additionally report **Tokens**, the mean total input and output tokens consumed by all model calls per case, expressed in thousands (K).

Figure 4 illustrates the scoring rules for the case annotated in Figure 3, with all three trajectories submitting the correct diagnosis (JRA = 1). Trajectory A establishes M_1, M_2, and M_3, completing the selector–label comparison to achieve MC = 1 and ECR = 1. Trajectory C reaches the same MC = 1 and ECR = 1 via the alternative configuration branch (M_1 and M_4). In contrast, Trajectory B establishes only M_1 and M_2 while omitting the backend-label check M_3; under $\phi_j^* = M_1 \wedge [M_4 \vee (M_2 \wedge M_3)]$, it receives partial coverage $\text{MC}_B = [1 + \max(0, (1 + 0)/2)]/2 = 0.75$ and ECR = 0. Across all trajectories, the gray steps represent extra exploratory calls that increase investigation length (Steps) and reduce efficiency (EE) without penalizing MC or ECR.

4.2 RQ1: RCA Outcome Effectiveness

Table 5 compares eleven models under the shared ReAct scaffold and the 20-step budget. GLM-5.1 achieves the highest JRA on OnlineBoutique (0.78), while GPT-5 leads on TrainTicket (0.68). The

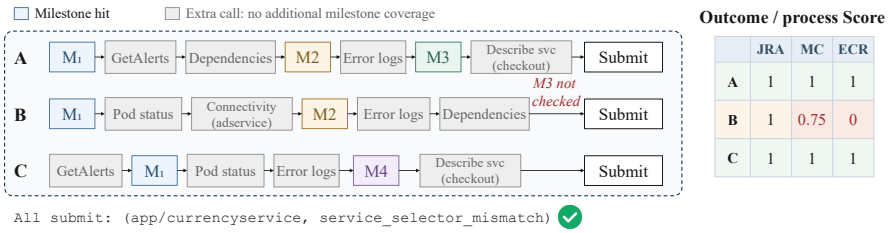

Fig. 4. Illustrative trajectories with the same correct RCA but different evidence coverage. A and C complete alternative evidence paths, whereas B omits backend-label evidence; gray steps indicate additional exploration that does not change MC or ECR.

relative performance of individual models also changes across workloads: GLM-5.1 reaches 0.53 JRA on TrainTicket, whereas GPT-5 obtains 0.68 on both systems. Open-weight models are competitive in this setting. DeepSeek-V4-Flash reaches 0.73 JRA on OnlineBoutique and 0.63 on TrainTicket. Qwen3.5-27B reaches 0.72 JRA on OnlineBoutique, compared with 0.68 for GPT-5 and 0.67 for Gemini-2.5-Pro. These results provide useful reference points for model selection; the component, fault-type, and difficulty breakdowns below show which aspects of diagnosis contribute to the aggregate scores.

(1) Locating the faulty component and identifying its fault mechanism present different challenges. On OnlineBoutique, every evaluated model achieves higher CA than FA. GPT-5 obtains 0.86 CA, 0.69 FA, and 0.68 JRA, while GLM-5.1 obtains 0.88 CA, 0.79 FA, and 0.78 JRA. For these models, approximately 18% and 10% of cases, respectively, have a correct component prediction but an incorrect or missing fault type. Agents can therefore locate the faulty component without correctly explaining what has gone wrong. Pod states, events, resource configurations, and application logs often identify an affected component directly. However, different faults in the same component can produce similar symptoms. Identifying the specific mechanism requires further inspection of the relevant configuration and runtime evidence.

TrainTicket also exposes errors in the opposite direction. Qwen3.5-Plus achieves 0.79 FA but 0.67 CA and 0.64 JRA. Approximately 15% of cases therefore have a correct fault type but an incorrect or missing component. TrainTicket includes gateways and shared database dependencies, allowing a fault in one component to produce errors in several others. For example, a database failure may appear in an application service's logs, requiring the agent to distinguish the service reporting the error from the dependency responsible for it. Recognizing the failure mechanism is insufficient when its source is attributed to the wrong component. JRA requires both parts of the diagnosis to be correct.

(2) Diagnostic accuracy declines across the predefined difficulty strata. Figure 5(a) presents JRA across the difficulty groups defined in Table 2. Several models achieve high accuracy on Easy faults, but their performance generally declines on the more difficult groups. On OnlineBoutique, GLM-5.1 achieves 0.91, 0.73, and 0.58 JRA across Easy, Medium, and Hard faults. DeepSeek-V4-Flash obtains 0.92, 0.72, and 0.57, while Qwen3.5-27B obtains 0.87, 0.71, and 0.41.

The difficulty increase is particularly pronounced on TrainTicket. GPT-5 achieves 0.88 JRA on Easy faults and 0.89 on Medium faults, but 0.00 on Hard faults. GLM-5.1 obtains 0.73, 0.56, and 0.00, respectively. Qwen3.5-Plus reaches the highest Hard-fault JRA of 0.29, followed by DeepSeek-V4-Flash at 0.22. Strong results on Easy and Medium faults therefore do not imply comparable success on performance-related incidents.

Easy faults often provide direct clues, including image-pull events, scheduling failures, and explicit runtime errors. These observations can narrow the diagnosis to a small set of components

Table 5. Performance comparison of LLM agents on outcome and process metrics.The best results in each metric are bold.

	OnlineBoutique									TrainTicket								
	Outcome			Process						Outcome			Process					
Model	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	RAR↓	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	RAR↓
GPT-5	0.86	0.69	0.68	0.70	0.62	0.39	**0.64**	**5.56**	**0.00**	**0.70**	0.71	**0.68**	0.72	0.68	0.40	**0.64**	7.03	**0.00**
GLM-5.1	**0.88**	**0.79**	**0.78**	**0.84**	0.74	**0.65**	0.54	7.75	**0.00**	0.65	0.58	0.53	0.65	0.63	0.35	0.38	8.83	0.02
Claude-Sonnet-4	0.82	0.67	0.66	0.82	0.75	0.64	0.41	9.47	0.01	0.63	0.64	0.62	**0.76**	0.71	0.15	0.52	7.42	**0.00**
Gemini-2.5-Pro	0.81	0.69	0.67	0.79	0.74	0.62	0.61	6.21	**0.00**	0.63	0.61	0.58	0.73	0.67	0.40	0.55	7.13	**0.00**
Qwen3.5-Plus	0.74	0.64	0.59	0.72	0.67	0.44	0.59	6.39	**0.00**	0.67	**0.79**	0.64	0.73	0.69	0.41	0.63	5.90	**0.00**
Qwen3-235B-A22B	0.62	0.45	0.39	0.55	0.47	0.34	0.33	7.22	0.02	0.47	0.27	0.26	0.62	0.60	0.33	0.34	8.67	0.03
DeepSeek-V4-Flash	0.86	0.74	0.73	0.80	0.71	0.57	0.58	7.24	**0.00**	0.69	0.71	0.63	**0.76**	**0.72**	**0.51**	0.57	7.74	**0.00**
Gemini-2.5-Flash	0.78	0.64	0.63	0.77	0.74	0.56	0.55	6.29	0.01	0.52	0.46	0.42	0.51	0.46	0.23	0.46	**4.95**	**0.00**
Qwen3.5-27B	0.86	0.73	0.72	0.82	**0.78**	0.64	0.56	6.97	0.01	0.63	0.61	0.56	0.71	0.68	0.42	0.57	7.03	0.01
Qwen3-14B	0.61	0.43	0.42	0.72	0.66	0.55	0.32	14.18	0.26	0.42	0.25	0.25	0.66	0.61	0.39	0.32	14.86	0.31
Qwen3-8B	0.38	0.22	0.21	0.57	0.45	0.19	0.34	15.78	0.45	0.43	0.30	0.26	0.62	0.61	0.26	0.40	11.50	0.22

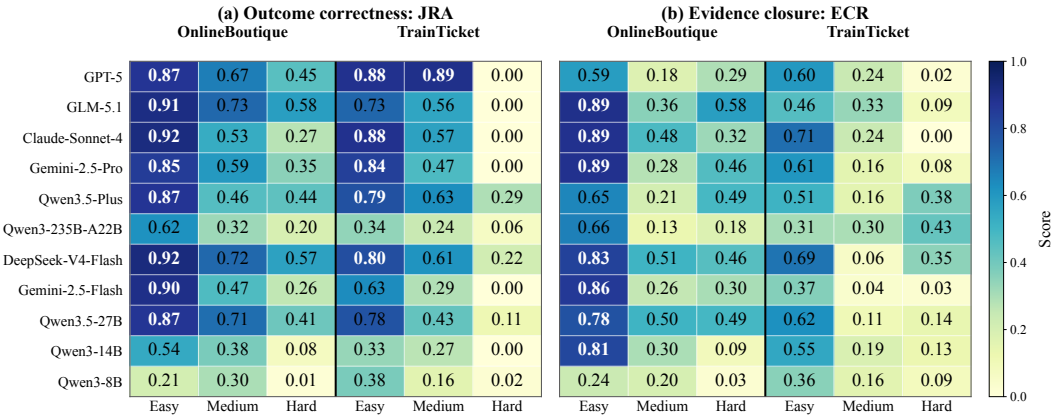

Fig. 5. JRA (a) and ECR (b) of LLM agents on OnlineBoutique and TrainTicket across Easy, Medium, and Hard faults.

and fault types. Medium faults require combining evidence across multiple components, Kubernetes resource objects, and system layers. Although several models handle these investigations well, Hard faults require distinguishing mechanisms that can produce similar performance symptoms. On OnlineBoutique, for example, resource saturation may result from a ChaosBlade-injected fault or from an application code defect. The agent must therefore examine the relevant source code alongside telemetry to distinguish these causes.

TrainTicket's Hard cases involve ChaosBlade-induced performance faults, where Pods and services can appear healthy and the injection itself leaves no explicit diagnostic clues. Locating these faults requires distinguishing the source of degradation from propagated symptoms across many services, a challenge central to telemetry-based RCA [41, 54]. Although `GetAlerts` provides threshold-based anomaly signals, these signals alone do not identify the root cause. These characteristics help explain the low accuracy on TrainTicket Hard cases. Despite strong results on simpler faults, the benchmark continues to distinguish model capabilities on Medium and especially Hard faults.

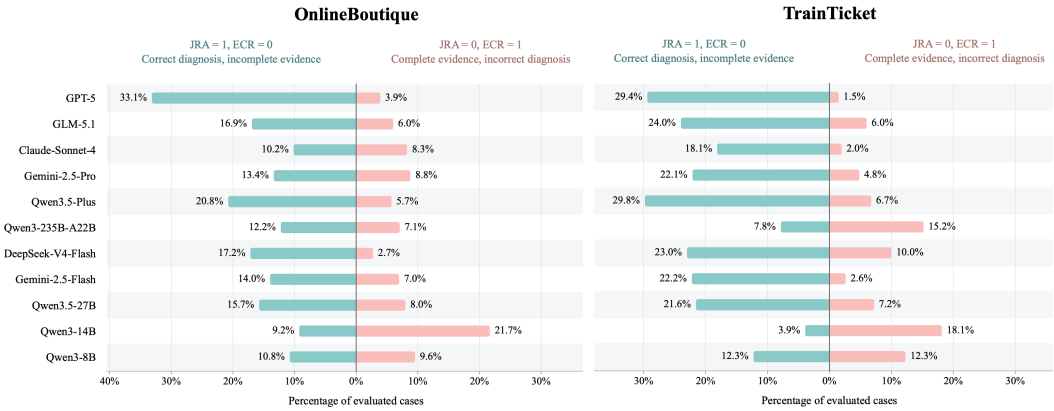

Fig. 6. Joint analysis of diagnostic correctness and evidence closure on OnlineBoutique and TrainTicket. Left bars show correct diagnoses with incomplete evidence (JRA=1, ECR=0), and right bars show complete evidence collection with incorrect diagnoses (JRA=0, ECR=1).

4.3 RQ2: RCA Process Alignment

We examine evidence completeness, acquisition order, and tool-use efficiency in the ReAct investigations reported in Table 5.

(1) Correct diagnoses and complete evidence collection do not always coincide. Evidence closure varies substantially across models. On OnlineBoutique, Qwen3.5-27B reaches 0.72 JRA and 0.64 ECR, while Gemini-2.5-Pro reaches 0.67 JRA and 0.62 ECR. These agents complete the evidence requirements in a majority of their investigations. GPT-5 shows a larger difference, with 0.68 JRA and 0.39 ECR on OnlineBoutique; on TrainTicket, it reaches 0.68 JRA and 0.40 ECR. The aggregate scores establish that accuracy and closure can differ, but the joint results show what happens within individual investigations.

Figure 6 separates the two directions of disagreement. For GPT-5, correct diagnoses with incomplete evidence account for 33.1% of OnlineBoutique cases and 29.4% of TrainTicket cases. The reverse pattern, complete evidence with an incorrect diagnosis, occurs in only 3.9% and 1.5%. Qwen3-14B shows the opposite balance: complete evidence with an incorrect diagnosis occurs in 21.7% and 18.1% of cases, exceeding its correct diagnoses with incomplete evidence (9.2% and 3.9%). Even for Qwen3.5-27B, whose aggregate JRA and ECR are close on OnlineBoutique, the two groups account for 15.7% and 8.0% of cases. A small difference between aggregate scores can therefore contain both kinds of disagreement.

Process scores clarify what an incomplete investigation has collected. GPT-5 reaches 0.70 MC but only 0.39 ECR on OnlineBoutique; Qwen3-14B reaches 0.66 MC and 0.39 ECR on TrainTicket. MC credits evidence acquired along an unfinished diagnostic path, whereas ECR requires a complete path. EOC adds the acquisition-order requirement: GPT-5's OnlineBoutique score falls from 0.70 MC to 0.62 EOC, and Qwen3-14B's TrainTicket score falls from 0.66 to 0.61. Some agents therefore stop before obtaining a required fact; others collect relevant facts before establishing their diagnostic prerequisites. Complete-but-incorrect investigations present a different problem: the required evidence was collected, but the final RCA was still wrong.

(2) Auditing outcome-correct but unclosed trajectories. To verify whether unclosed trajectories reflect genuine investigative gaps rather than unannotated alternative paths in $\mathcal{D}_j^*$, we manually audited a random sample of 100 trajectories from GPT-5, DeepSeek-V4-Flash and Qwen3.5-27B.

Table 6. Comparison of experience reuse methods for RCA agents. The best results in each metric are bold.

		OnlineBoutique								TrainTicket							
Model	**Method**	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓
DeepSeek-V4-Flash	Base	0.88	0.76	0.76	0.83	0.75	0.64	**0.63**	6.41	**0.69**	**0.71**	**0.63**	**0.76**	0.72	**0.50**	**0.55**	7.68
	ICL	0.90	**0.83**	**0.83**	**0.91**	**0.87**	**0.78**	**0.63**	6.75	0.63	0.61	0.55	**0.76**	**0.74**	0.48	0.50	7.08
	SOP	**0.91**	**0.83**	0.82	0.86	0.77	0.70	0.46	**6.25**	0.66	0.64	0.61	0.75	0.70	0.48	0.43	**5.93**
Gemini-2.5-Flash	Base	0.82	0.67	0.65	0.79	0.75	0.60	**0.58**	**6.49**	0.60	0.55	0.53	0.57	0.54	0.30	**0.53**	4.91
	ICL	**0.84**	**0.74**	**0.74**	**0.84**	**0.81**	**0.69**	0.54	6.88	0.59	0.60	0.58	**0.73**	**0.71**	**0.48**	0.49	6.87
	SOP	**0.84**	0.71	0.67	0.81	0.76	0.64	0.41	**6.49**	**0.68**	**0.64**	**0.61**	0.69	0.64	0.39	0.45	**4.86**
Qwen3.5-27B	Base	0.85	0.71	0.70	0.81	0.77	0.61	0.53	**6.87**	0.62	**0.64**	0.58	**0.72**	**0.70**	0.41	**0.55**	7.05
	ICL	**0.88**	**0.79**	**0.79**	**0.91**	**0.87**	**0.79**	**0.61**	7.35	0.61	0.59	0.54	0.70	0.69	**0.42**	0.45	6.96
	SOP	0.82	0.64	0.63	0.79	0.73	0.57	0.38	7.17	**0.73**	0.62	**0.59**	0.67	0.61	0.38	0.30	**6.41**

Overall, **95%** of the audited cases stem from incomplete agent investigations. Specifically, Symptom-log termination without source tracing (73%) accounts for the majority, where agents stopped immediately after observing a discriminative error log or event without tracing backward to verify the underlying configuration or resource state. While such log cues often correlate strongly with the fault label to yield a correct prediction, stopping at symptom logs leaves the concrete faulty artifact unverified for downstream remediation. Furthermore, Inconclusive evidence and heuristic guessing (22%) involves cases where agents terminated with partial observations that remained logically ambiguous and could not rule out competing fault mechanisms. Only the remaining unannotated alternative paths (5%) established the root cause through valid indirect observations outside our direct evidence bundles. Even if this 5% margin were credited as fully closed, aggregate ECR scores would shift by less than 0.02, confirming that the outcome–process gap primarily reflects premature diagnostic termination rather than annotation false negatives.

(3) Longer investigations do not necessarily produce more complete evidence. On OnlineBoutique, Qwen3-8B averages 15.78 steps with 0.34 EE and 0.19 ECR. Its RAR of 0.45 shows that many calls repeat earlier tool-and-argument pairs without adding evidence or advancing a new milestone. Qwen3-14B also conducts relatively long investigations, averaging 14.18 steps with 0.32 EE and 0.26 RAR, although it reaches a higher ECR of 0.55. Repeated calls also occur on TrainTicket, where these models have RAR values of 0.22 and 0.31, respectively. Continuing an investigation does not ensure that its remaining evidence gaps are addressed.

Shorter investigations can leave evidence incomplete for a different reason. GPT-5 averages 5.56 steps on OnlineBoutique, with 0.64 EE and a near-zero RAR, but reaches only 0.39 ECR. Many of its calls contribute evidence, yet the investigation often ends without completing the required checks. GLM-5.1 averages 7.75 steps with 0.54 EE and 0.65 ECR, also with a near-zero RAR. A higher fraction of evidence-contributing calls therefore does not necessarily mean that more investigations collect complete evidence.

These results show two different patterns of incomplete investigation. Some agents finish after collecting useful but insufficient evidence. Others continue calling tools but repeat checks that do not supply the missing facts. Investigation length alone does not distinguish these behaviors or establish whether the diagnosis is fully supported.

4.4 RQ3: Experience-Augmented RCA Agents

In real cloud operations, engineers draw on prior incident traces, on-call notes, and runbooks to guide diagnosis. We therefore examine whether reusable operational experience can improve RCA agents. For each root cause, we partition the cases into disjoint training and test sets at an approximately 1:1 ratio. The experience pool contains only successful GPT-5 trajectories from the training set, and Base, ICL, and SOP are evaluated on the same held-out test cases.

We compare two forms of experience reuse. *Trajectory-as-ICL*[5] retrieves three successful trajectories matched by the reported symptom, whereas *Standard Operating Procedure (SOP)*[14, 40] distills recurring diagnostic actions and decision branches into category-level runbook guidance. The target root cause is unknown during diagnosis, so neither method uses the ground-truth root-cause label to select experience for a test incident. We select Gemini-2.5-Flash, Qwen3.5-27B, and DeepSeek-V4-Flash because they combine competitive base performance with relatively low inference cost and latency. Table 6 reports the results.

Historical experience can improve diagnosis, but its benefits vary across models, systems, and reuse methods. On OnlineBoutique, ICL improves both JRA and ECR for all three models. Averaged across models, JRA increases from 0.70 to 0.79, and ECR increases from 0.62 to 0.75. SOP produces smaller average gains, reaching 0.71 JRA and 0.64 ECR, with different effects across models. It improves both metrics for DeepSeek-V4-Flash and Gemini-2.5-Flash, but reduces Qwen3.5-27B's JRA from 0.70 to 0.63 and ECR from 0.61 to 0.57. Supplying successful historical experience therefore does not guarantee an improvement for every model.

The TrainTicket results are more mixed. Average JRA changes from 0.58 for Base to 0.56 with ICL and 0.60 with SOP, while average ECR changes from 0.40 to 0.46 and 0.42, respectively. ICL improves Gemini-2.5-Flash's JRA from 0.53 to 0.58 and ECR from 0.30 to 0.48, but reduces both metrics for DeepSeek-V4-Flash. Accuracy and evidence completeness also respond differently to the same guidance. For Gemini-2.5-Flash, SOP achieves higher JRA than ICL (0.61 versus 0.58), but lower ECR (0.39 versus 0.48). The method that produces more correct diagnoses therefore need not produce more investigations with complete supporting evidence.

A practical difficulty is selecting relevant experience before the root cause is known. Similar symptoms can arise from different mechanisms, so a retrieved successful trajectory may describe a different cause and omit checks needed for the current incident. ICL provides detailed examples but is limited to three retrieved trajectories. SOP covers recurring branches within a fault category, but the agent must still determine which branches apply. These differences may contribute to the uneven gains across models and systems.

These results suggest that the effectiveness of experience reuse depends not only on the availability of prior successful diagnoses, but also on how that experience is represented, retrieved, and matched to the current incident. The incident reports, root-cause annotations, and snapshot-preserved system observations in Cloud-OpsBench provide a basis for studying more adaptive mechanisms for organizing and retrieving diagnostic experience.

Table 7. Comparison of agent architectures and harnesses using GLM-5.1. The best results in each metric are bold.

	OnlineBoutique									TrainTicket								
Method	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	Tokens (K)↓	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	Tokens (K)↓
Agent architectures																		
ReAct	0.93	0.84	0.81	0.86	0.76	0.68	0.57	7.44	**55.13**	0.62	0.60	0.55	0.65	0.63	0.38	0.35	10.17	129.46
Plan-and-Execute	0.90	0.77	0.76	0.84	0.75	0.64	0.52	7.98	60.47	0.72	0.65	0.62	0.77	0.73	0.50	**0.48**	**9.85**	**126.77**
Reflection	0.94	0.85	0.84	0.87	0.75	0.68	**0.62**	**7.41**	76.82	0.69	0.60	0.57	0.67	0.64	0.36	0.44	11.69	230.78
Agent harnesses																		
Codex	**0.95**	**0.87**	**0.86**	**0.94**	0.76	**0.83**	0.39	11.87	126.29	**0.80**	**0.69**	**0.67**	**0.91**	**0.90**	**0.81**	0.38	16.91	205.17
OpenCode	0.94	0.84	0.84	0.93	**0.79**	0.81	0.42	11.71	105.85	0.68	0.66	0.63	0.86	0.82	0.71	0.42	21.95	361.27

Note: Planning and control calls are excluded from step counts.

4.5 RQ4: Agent Architectures and Harnesses

The preceding experiments compare models and experience reuse within a ReAct-style loop. Here, we hold the base model fixed at GLM-5.1 and compare five ways of organizing the investigation:

ReAct[56], Plan-and-Execute[49], Reflection[44], Codex, and OpenCode. Plan-and-Execute uses a planner to select diagnostic subgoals and a ReAct executor to investigate them, with replanning based on the accumulated evidence until the planner submits the final RCA. Reflection reviews a candidate diagnosis against the collected evidence and permits further investigation and answer refinement. We also evaluate two coding-agent harnesses, Codex CLI and OpenCode,[2] which provide tool-mediated execution, context management, and iterative investigation workflows. We connect these harnesses to the benchmark's diagnostic tools through Model Context Protocol (MCP).

In this RQ, we increase the diagnostic tool-call budget from 20 in the main experiments to 50 to accommodate the additional diagnostic checks introduced by planning, orchestration, and recovery workflows [19], and apply the same budget to all five methods. Planning and control calls are excluded from this budget. Table 7 reports results over all cases. To compare the computational costs of these agent implementations, we additionally report mean end-to-end input and output token consumption per case, expressed in thousands (K).

(1) Similar diagnostic accuracy can accompany different levels of evidence completeness. On OnlineBoutique, Reflection and OpenCode both achieve 0.84 JRA, but their ECR values are 0.68 and 0.81, respectively. On TrainTicket, Plan-and-Execute and OpenCode achieve similar JRA values of 0.62 and 0.63, while their ECR values differ more substantially, at 0.50 and 0.71. Thus, agents that correctly diagnose similar proportions of cases can differ in how often they collect complete supporting evidence.

Codex achieves the highest JRA and ECR on both systems in this experiment, reaching 0.86 and 0.83 on OnlineBoutique, and 0.67 and 0.81 on TrainTicket. Its investigations also use more diagnostic steps than ReAct, Plan-and-Execute, and Reflection. The results indicate that the organization and continuation of an investigation affect how much evidence an agent collects, even when the underlying model and diagnostic tool-call budget are held fixed.

(2) More complete investigations consume more tokens, but higher expenditure does not guarantee better results. On OnlineBoutique, Codex uses 126.29K tokens per case, approximately 2.29 times ReAct's 55.13K. Its ECR increases from ReAct's 0.68 to 0.83, while JRA increases from 0.81 to 0.86. The additional computation accompanies improvements in both metrics, with a larger increase in evidence closure.

On TrainTicket, however, OpenCode uses 21.95 steps and 361.27K tokens per case, compared with Codex's 16.91 steps and 205.17K tokens, yet achieves lower JRA (0.63 versus 0.67) and ECR (0.71 versus 0.81). Codex therefore achieves both higher diagnostic accuracy and more complete evidence collection at a lower token cost in this comparison.

4.6 Analysis of RCA Agent Diagnostic Processes

We inspect recorded agent trajectories to examine how evidence acquisition and interpretation shape diagnostic outcomes. The analysis covers both incorrect diagnoses and correct diagnoses with incomplete evidence collection. The selected trajectories illustrate representative behaviors rather than estimate their prevalence.

4.6.1 Failure Modes in Incorrect Diagnoses. To understand agent errors beyond aggregate scores, we first collect failure cases with JRA = 0 from the 11 evaluated LLMs. From this failure pool, we qualitatively inspect a purposive sample of trajectories covering both benchmark systems, all eight fault categories, and models with different capability levels. This exploratory analysis characterizes representative and potentially overlapping behaviors rather than estimating their prevalence or

[2]We use Codex CLI v0.139.0 and OpenCode v1.18.31.

defining an exhaustive taxonomy. We organize the observed failures around four broad forms of diagnostic breakdown.

(1) Evidence-grounding failures. Agents sometimes retrieve useful evidence but interpret it incorrectly or assign it insufficient weight. For example, startup events may already report `manifest unknown`, `403 Forbidden`, or a missing secret, yet the agent continues checking peripheral resources. Agents can also anchor on salient downstream symptoms, such as frontend 503 responses, high latency, or a prominent CPU alert, and mistake the noisiest component for the causal source. Conversely, missing objects and telemetry can be overlooked: an empty endpoint set, an absent pod, or the lack of logs from a failed component is sometimes treated as no evidence rather than as a diagnostic signal. Such misinterpretations can place the investigation in the wrong fault family, for example by explaining a code defect as OOM, network delay, or service misconfiguration.

(2) Search-control failures. Agents do not always adapt their investigation to the evidence already established. Some commit to a plausible hypothesis after inspecting one abnormal pod or service and stop before checking competing explanations. Others fail to redirect the search when an initial path is weakened; for instance, after confirming normal service connectivity, they may not move to gateway routing or configuration checks. The opposite behavior also occurs: even after observing decisive evidence such as `replicas=0`, an inactive `kube-proxy`, or an explicit database access error, an agent may continue issuing low-value or repeated queries. These trajectories reflect weak control over hypothesis revision, diagnostic branching, and stopping.

(3) Target-resolution failures. Relevant evidence does not necessarily lead agents to the correct diagnostic target. In TrainTicket, database errors observed by application services may be attributed to those services instead of the shared `tsdb-mysql` dependency; similarly, gateway routing faults may be assigned to affected business services. Agents also confuse closely related mechanisms, such as image-reference and registry-DNS failures, probe port and protocol errors, or adjacent code-defect subtypes. Codedefect cases further require connecting service-level alerts, HTTP errors, and abnormal metrics to source-level behavior. Agents may inspect relevant telemetry or code artifacts but still explain a busy loop, artificial delay, or excessive file operation as resource saturation or a performance fault.

(4) Tool-grounding failures. Failures also arise at the interface between reasoning and execution. Invalid parameters, an incorrect object or scope, or an action inconsistent with the intended check can prevent the required evidence from being obtained. More subtly, a syntactically valid but mis-specified query can create misleading evidence. For example, probing service connectivity with the wrong port produces a genuine connection failure that the agent may then treat as a production symptom and use to support an incorrect diagnosis.

Our process-oriented analysis reveals that agent failures often develop before the final prediction, through breakdowns in evidence-grounded investigation and reasoning. These observations show that improving current RCA agents requires not only better final predictions, but also more reliable and interpretable diagnostic trajectories.

4.6.2 Case Studies: Evidence Gaps in Correct Diagnoses. We examine three representative investigations in which the agent correctly identifies both the faulty component and root cause but receives ECR = 0. For each case, we compare the collected observations with the evidence available in the frozen snapshot to identify what remains unverified. These cases illustrate recurring forms of evidence incompleteness rather than model-specific failures or their prevalence.

(1) Missing backend-label evidence. The incomplete comparison illustrated in Fig. 4 also appears in actual agent trajectories. In OnlineBoutique `service/38`, a representative run observes that `shippingservice` has no endpoints and selects Pods with `app=shipping_service`. The agent correctly reports a selector mismatch but never retrieves the backend Pod labels or

equivalent configuration evidence. The frozen snapshot shows that the backend Pod is actually labeled `app=shippingservice`, but this fact is absent from the collected observations. The diagnosis is therefore correct, yet the selector–label mismatch itself is never directly verified, resulting in MC = 0.75 and ECR = 0.

(2) Testing a connection without checking the exposed port. In TrainTicket `runtime/58`, a representative trajectory correctly diagnoses an invalid MySQL port in `ts-auth-service`. The application configuration points to `tsdb-mysql-leader:34001`; the agent observes connection-refused errors and confirms that this address is unreachable. However, it never inspects the database Service or Endpoints to determine the port actually exposed. The frozen snapshot shows that the Service exposes port `3306`, but this fact is absent from the collected observations. The connectivity test therefore confirms failure at the configured address without establishing the port mismatch itself, leaving the investigation at MC = 0.75 and ECR = 0.

(3) Attributing latency to code without inspecting the implementation. In OnlineBoutique `codedefect/69`, The agent correctly reports an artificial delay in `productcatalogservice`. It retrieves alerts, Pod status, error logs, and application YAML. The alerts show that the service's p90 latency increases from 2.13 ms to 4003.98 ms, while the error-log query returns no relevant errors. However, the agent never invokes `ListCodeFiles` or `GetSourceCode`. The frozen snapshot contains an explicit three-second sleep in `ListProducts`, but this implementation evidence is never retrieved. The trajectory therefore establishes the latency symptom without verifying the code-level mechanism, resulting in MC = 1/3 and ECR = 0.

Across the three cases, the submitted RCA labels are correct, yet key supporting facts remain uncollected. These examples demonstrate that outcome correctness alone cannot distinguish a fully grounded diagnosis from one inferred from partial but salient evidence.

5 Discussion

5.1 Implications and Research Utility

Cloud-OpsBench is an evaluation infrastructure for interactive and evidence-grounded cloud RCA. Its contribution extends beyond a collection of cloud fault cases by making both the final diagnosis and its supporting investigation controlled and measurable.

Controlled experiments on cloud diagnosis. By replaying the same runtime-verified incident, Cloud-OpsBench allows researchers to compare RCA agents or vary one agent component while holding the available fault evidence fixed. For example, planners, memory mechanisms, topology and knowledge retrieval, and tool-selection policies can be compared using the same user query and incident state. Differences in diagnostic correctness, milestone coverage, evidence-order consistency, and investigation efficiency can therefore be attributed more directly to agent design rather than to a different realization of the fault. Unlike static telemetry inputs, the replay interfaces preserve the agent's decisions about which diagnostic tool, component, and evidence source to inspect.

Process-level analysis of RCA agents. The diagnostic evidence graph aligns different tool-mediated investigations according to the causal facts they establish. For a failure that propagates across services or platform resources, it can show whether an agent observes the reported manifestation but fails to traverse the relevant dependency, reaches the faulty component without confirming its mechanism, or concludes before acquiring sufficient evidence. Alternative evidence bundles and partial-order dependencies allow these failures to be analyzed without requiring agents to reproduce a canonical command sequence. This enables targeted studies of whether topology information improves propagation tracing, retrieved operational knowledge improves mechanism confirmation, or memory and planning reduce repeated or causally premature investigation.

Reusable diagnostic experience. Trajectories produced on Cloud-OpsBench can be aligned to the same milestones even when agents follow different investigation paths. This alignment

makes successful and failed strategies comparable across models, fault types, and application and platform layers, providing a basis for studying how diagnostic experience transfers across incidents. Although this work focuses on evaluation, the resulting trajectories can also support future research on experience retrieval, diagnostic skill distillation, and process-aware training for cloud RCA agents.

5.2 Future Research Opportunities

State-adaptive evidence acquisition. Current RCA agents often rely on generic investigation routines that do not adapt sufficiently to the evidence already established. Future work can study policies that select the next tool and target according to the grounded milestones, unresolved diagnostic alternatives, and expected information gain. An important question is whether topology-aware planning and uncertainty-aware stopping can improve evidential completeness while avoiding premature conclusions and unnecessary inspection.

Transferable diagnostic skills. Cloud incidents differ in their components and topologies, but often require recurring diagnostic operations such as dependency traversal, failure-boundary localization, and mechanism confirmation. Aligning trajectories through diagnostic milestones provides a basis for learning or retrieving skills that are invariant to component identifiers and exact command syntax. Future studies should examine whether experience acquired from one workload or fault type improves investigation of unseen incidents without relying on root-cause labels or memorized case-specific paths.

RCA under incomplete and interacting evidence. The current benchmark focuses on individually verified faults whose required evidence is captured in the verified incident state or its associated case-specific incident interval. Operational incidents may instead involve concurrent faults, delayed effects, changing system states, and missing or contradictory observations. Extending the construction pipeline to such settings would enable research on maintaining competing hypotheses, distinguishing root causes from propagated symptoms, and expressing uncertainty when the available evidence does not support a unique diagnosis. Such extensions should preserve matched evaluation conditions and auditable evidence requirements rather than introducing complexity at the expense of comparability.

5.3 Threats to Validity

External Validity. Cloud-OpsBench focuses on Kubernetes-based cloud-native systems, which may limit generalization to other architectures and operational environments. We mitigate this threat by covering 57 fault types drawn from multiple sources, two workloads with different service scales, both application and platform layers, and retaining only runtime-verified cases. Nevertheless, broader validation requires additional workloads, deployment configurations, and real-world incidents. Each benchmark release also reflects the Kubernetes version used during fault realization and snapshot capture. Most supported diagnostic interfaces rely on stable API resources and long-standing troubleshooting operations governed by Kubernetes compatibility and deprecation policies [27], which reduces drift under routine upgrades. When a Kubernetes update changes a fault manifestation or diagnostic interface, the construction pipeline can re-execute the affected cases and recapture their snapshots as a revalidated, versioned release.

Internal Validity. Model comparisons may be affected by the agent scaffold, prompting choices, inference nondeterminism, and differences in prior exposure to Kubernetes knowledge. We mitigate these factors by using the same ReAct framework, prompt structure, tool schemas, step budget, temperature setting, and snapshot evidence across models. Because process credit requires case-specific observations acquired through interaction, prior domain knowledge alone cannot obtain a complete process score.

Construct Validity. Cloud-OpsBench evaluates diagnosis from evidence preserved in the verified incident state and its associated case-specific incident interval rather than from arbitrary system history or subsequent live state transitions. We mitigate this limitation by covering the principal evidence channels used in cloud-native diagnosis and retaining only cases whose required milestones are observable and replay-validated. Faults whose diagnosis requires long-term historical baselines, evidence preceding baseline confirmation, or observations produced only after verification are outside the current benchmark scope.

Furthermore, our process metrics require positive observational verification of the underlying faulty state and mechanism. In principle, an agent may predict the correct label by relying on salient downstream error logs or indirect elimination without directly inspecting the faulty configuration or resource. Under our scoring, such trajectories receive partial Milestone Coverage (MC) credit for the collected clues but ECR = 0. We intentionally maintain this requirement because actionable production RCA requires directly verifying the root-cause artifact before remediation, rather than relying on unverified symptom correlations or elimination alone.

6 Related Work

Evolution of RCA. Traditional AIOps RCA formulates diagnosis as discriminative localization over metrics, logs, and traces, ranking faulty components with causal or graph models [20, 29, 30, 33, 34, 57–59, 67]. LLM-based systems then treat RCA as generative analysis over a provided incident context, often retrieving operational documents to produce explanations [1, 9, 15, 32, 53]. Recent agentic workflows go further: they select diagnostic tools, interpret returned observations, and revise hypotheses through multi-step interaction with cloud and related operational environments [18, 38, 40, 48, 50, 55, 65, 66]. This shift makes interactive evidence acquisition central to RCA and motivates a new class of benchmarks that can evaluate not only final root-cause labels, but also tool-mediated investigation under controlled, comparable incident conditions.

Benchmarking in AIOps. Existing AIOps benchmarks only partially meet this need. Static telemetry benchmarks provide deterministic offline artifacts for detection or localization, but agents cannot investigate through operational interfaces [2, 3, 11, 29, 41, 45, 59, 61, 69]. OpenRCA allows code-based queries over supplied files, which improves inspectability without exposing system-facing SRE tools [54]. Live-environment benchmarks restore standard tool use on real services, but fault manifestations can vary across runs, which complicates controlled comparison and fine-grained process scoring [8, 10, 22, 47]. Across both paradigms, evaluation remains largely outcome-oriented. As summarized in Table 1, Cloud-OpsBench targets the missing combination: snapshot-backed replay preserves system-facing investigation under matched incident observations, and diagnostic evidence graphs score whether required facts are established in an admissible order from tool observations, rather than only whether the final root-cause label is correct or whether the agent reproduces a single expert command sequence.

Trajectory and Process Evaluation of LLM Agents. Recent studies have explored evaluating intermediate agent trajectories across reasoning, web, and desktop environments. In mathematical reasoning, process supervision [35] trains reward models to score the validity of individual derivation steps. For interactive task execution, AgentBoard [37] tracks fine-grained progress through environment states and subgoal completion, while WebCanvas [39] evaluates web agents through critical intermediate actions and states. CocoaBench [19] evaluates long-horizon digital-agent workflows using task-specific automatic evaluators, while OSWorld-Pro [52] decomposes computer-use tasks into sequentially dependent subgoals and employs LLM-based judges [63] for process-level assessment. However, these paradigms primarily evaluate reasoning-step correctness, task progress, or observable execution outcomes, rather than diagnostic evidence acquisition. In contrast, cloud-native RCA is a read-only diagnostic probing task: agents query a distributed system

without mutating its state, and the same diagnostic fact can be uncovered through multiple tool channels, query scopes, and non-linear investigation orders. While OpenRCA 2.0 [13] introduced PAVE to reconstruct fault-propagation paths using fault-injection information and telemetry, Cloud-OpsBench formalizes deterministic process evaluation over interactive tool observations, coupling conjunctive and disjunctive completion logic and prerequisite dependencies with exhaustively enumerated snapshot evidence bundles.

7 Conclusion

This paper presents Cloud-OpsBench, an evaluation infrastructure for interactive and evidence-grounded cloud RCA. It combines runtime-verified fault realization with snapshot-backed replay, allowing agents to investigate matched incident observations through standard diagnostic interfaces while independently selecting their tools and targets. Each case pairs an outcome label with a diagnostic evidence graph, enabling evaluation of root-cause correctness, milestone coverage, evidence-order consistency, and investigation efficiency without prescribing a canonical command sequence. With 754 cases spanning 57 fault types across application and Kubernetes platform layers, Cloud-OpsBench provides a controlled basis for comparing RCA agents and analyzing where their investigations fail in evidence acquisition, dependency traversal, mechanism confirmation, and final diagnosis.

References

[1] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In *ICSE 2023*. IEEE, 1737–1749. doi:10.1109/ICSE48619.2023.00149

[2] AIOPS.CN. 2020. AIOPS 2020 challenges data. https://github.com/NetManAIOps/AIOps-Challenge-2020-Data.

[3] AIOPS.CN. 2021. AIOPS 2021 challenges data. https://www.aiops.cn/gitlab/aiops-nankai/data/trace/aiops2021.

[4] Marco Barletta, Marcello Cinque, Catello Di Martino, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2024. Mutiny! How Does Kubernetes Fail, and What Can We Do About It?. In *DSN 2024*. IEEE, 1–14. doi:10.1109/DSN58291.2024.00016

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[6] Chaosblade. 2026. Chaosblade. https://chaosblade.io/.

[7] Junjie Chen, Shu Zhang, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Yu Kang, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2020. How Incidental are the Incidents? Characterizing and Prioritizing Incidents for Large-Scale Online Service Systems. In *ASE 2020*. IEEE, 373–384. doi:10.1145/3324884.3416624

[8] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *MLSys 2025*. https://openreview.net/forum?id=3EXBLwGxtq

[9] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *EuroSys 2024*. ACM, 674–688. doi:10.1145/3627703.3629553

[10] Jackson Clark, Yiming Su, Saad Mohammad Rafid Pial, Yifang Tian, Lily Gniedziejko, Hans-Arno Jacobsen, Yinfang Chen, and Tianyin Xu. 2026. SREGym: A Live Benchmark for AI SRE Agents with High-Fidelity Failure Scenarios. *arXiv preprint arXiv:2605.07161* (2026).

[11] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Chenyu Zhao, Shimin Tao, Yilun Liu, Shenglin Zhang, Duoming Lin, Changchang Liu, Yuzhe Cai, Weibin Meng, Yongqian Sun, and Dan Pei. 2025. LogEval: A comprehensive benchmark suite for LLMs in log analysis. *Empir. Softw. Eng.* 30, 6 (2025), 173. doi:10.1007/S10664-025-10701-6

[12] Rob Ewaschuk. 2016. Monitoring Distributed Systems. In *Site Reliability Engineering: How Google Runs Production Systems*, Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). O'Reilly Media, Chapter 6. https://sre.google/sre-book/monitoring-distributed-systems/

[13] Aoyang Fang, Yifan Yang, Jin'ao Shang, Qisheng Lu, Junjielung Xu, Rui Wang, Songhan Zhang, Yuzhong Zhang, Boxi Yu, and Pinjia He. 2026. OpenRCA 2.0: From Outcome Labels to Causal Process Supervision. *arXiv preprint arXiv:2606.27154* (2026).

[14] Yufei Gao, Zhengong Cai, and Bowei Yang. 2026. RCAFlow: A Workflow-Informed Hierarchical Planning Multi-Agent System for Root Cause Analysis. In *AAAI 2026*, Vol. 40. 300–308. doi:10.1609/aaai.v40i1.36991

[15] Drishti Goel, Fiza Husain, Aditya Singh, Supriyo Ghosh, Anjaly Parayil, Chetan Bansal, Xuchao Zhang, and Saravan Rajmohan. 2024. X-Lifecycle Learning for Cloud Incident Management using LLMs. In *FSE Companion 2024*. ACM, 417–428. doi:10.1145/3663529.3663861

[16] Albert Gong, Kyuseong Choi, Abhineet Agarwal, Jason Schechner, Ryan Huang, Raj Agrawal, Anish Agarwal, and Raaz Dwivedi. 2026. ORCA-bench: How Ready Are Language Model Agents for Oncall? *arXiv preprint arXiv:2607.28545* (2026).

[17] Google. 2026. Online Boutique: Sample cloud-first application with 10 microservices showcasing Kubernetes, Istio, and gRPC. https://github.com/GoogleCloudPlatform/microservices-demo.

[18] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model Based Multi-agents: A Survey of Progress and Challenges. In *IJCAI 2024*. ijcai.org, 8048–8057. https://www.ijcai.org/proceedings/2024/890

[19] Shibo Hao, Zhining Zhang, Zhiqi Liang, Tianyang Liu, Yuheng Zha, Qiyue Gao, Jixuan Chen, Zilong Wang, Zhoujun Cheng, Haoxiang Zhang, et al. 2026. CocoaBench: Evaluating unified digital agents in the wild. In *Third Conference on Language Modeling*.

[20] Jin Huang, Pengfei Chen, Guangba Yu, Yilun Wang, Haiyu Huang, and Zilong He. 2024. FaaSRCA: Full Lifecycle Root Cause Analysis for Serverless Applications. In *ISSRE 2024*. IEEE, 415–426. doi:10.1109/ISSRE62328.2024.00047

[21] Istio. 2026. Service Mesh. Simplified. https://istio.io/.

[22] Saurabh Jha, Rohan R. Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, Noah Zheutlin, Saki Takano, Divya Pathak, Felix George, Xinbo Wu, Bekir O. Turkkan, Gerard Vanloo, Michael Nidd, Ting Dai, Oishik Chatterjee, Pranjal Gupta, Suranjana Samanta, Pooja Aggarwal, Rong Lee, Jae-wook Ahn, Debanjana Kar, Amit M. Paradkar, Yu Deng, Pratibha Moogi, Prateeti Mohapatra, Naoki Abe, Chandrasekhar Narayanaswami, Tianyin Xu, Lav R. Varshney, Ruchi Mahindru, Anca Sailer, Laura Shwartz, Daby Sow, Nicholas C. Fuller, and Ruchir Puri. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *ICML 2025*. OpenReview.net. https://openreview.net/forum?id=jP59rz1bZk

[23] Chris Jones. 2016. Effective Troubleshooting. In *Site Reliability Engineering: How Google Runs Production Systems*, Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). O'Reilly Media, Chapter 12. https://sre.google/sre-book/effective-troubleshooting/

[24] k8sdoc. 2025. Kubernetes Documentation. Retrieved June 10, 2026 from https://kubernetes.io/docs/concepts/

[25] NVJK Kartik, Garvit Sapra, Rishav Hada, and Nikhil Pareek. 2025. AgentCompass: Towards Reliable Evaluation of Agentic Workflows in Production. arXiv:2509.14647 [cs.AI] https://arxiv.org/abs/2509.14647

[26] KodeKloud. 2025. KodeKloud Notes. Retrieved June 10, 2026 from https://notes.kodekloud.com/

[27] Kubernetes Authors. 2024. Kubernetes Deprecation Policy. Retrieved Jul. 13, 2026 from https://kubernetes.io/docs/reference/using-api/deprecation-policy/

[28] LearnKube.CN. 2025. LearnKUBE. https://learnkube.com/troubleshooting-deployments.

[29] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.

[30] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. 2023. SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. *IEEE Trans. Dependable Secur. Comput.* 20, 4 (2023), 2762–2780. doi:10.1109/TDSC.2022.3162857

[31] Xiaoyun Li, Guangba Yu, Pengfei Chen, Hongyang Chen, and Zhekang Chen. 2022. Going through the Life Cycle of Faults in Clouds: Guidelines on Fault Handling. In *ISSRE 2022*. IEEE, 121–132. doi:10.1109/ISSRE55969.2022.00022

[32] Yichen Li, Yulun Wu, Jinyang Liu, Zhihan Jiang, Zhuangbin Chen, Guangba Yu, and Michael R. Lyu. 2025. COCA: Generative Root Cause Analysis for Distributed Systems with Code Knowledge. In *ICSE 2025*. IEEE, 1346–1358. doi:10.1109/ICSE55347.2025.00234

[33] Yufeng Li, Guangba Yu, Pengfei Chen, Chuanfu Zhang, and Zibin Zheng. 2022. MicroSketch: Lightweight and Adaptive Sketch Based Performance Issue Detection Localization in Microservice Systems. In *ICSOC 2022*. Springer-Verlag, Berlin, Heidelberg, 219–236. doi:10.1007/978-3-031-20984-0_15

[34] Zeyan Li, Nengwen Zhao, Mingjie Li, Xianglin Lu, Lixin Wang, Dongdong Chang, Xiaohui Nie, Li Cao, Wenchi Zhang, Kaixin Sui, Yanhua Wang, Xu Du, Guoqiang Duan, and Dan Pei. 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In *ESEC/FSE 2022*. ACM, 996–1008. doi:10.1145/3540250.3549092

[35] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://openreview.net/forum?id=v8L0pN6EOi

[36] Locust. 2026. An open source load testing tool. https://locust.io.
[37] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. Agentboard: An analytical evaluation board of multi-turn llm agents. *Advances in neural information processing systems* 37 (2024), 74325–74362.
[38] Jiayi Mao, Liqun Li, Yanjie Gao, Zegang Peng, Shilin He, Chaoyun Zhang, Si Qin, Samia Khalid, Qingwei Lin, Saravan Rajmohan, Sitaram Lanka, and Dongmei Zhang. 2026. StepFly: Agentic Troubleshooting Guide Automation for Incident Diagnosis. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE136 (June 2026), 23 pages. doi:10.1145/3808143
[39] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and Zhengyang Wu. 2024. WebCanvas: Benchmarking Web Agents in Online Environments. arXiv:2406.12373 [cs.CL] https://arxiv.org/abs/2406.12373
[40] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, Gaogang Xie, and Dan Pei. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *WWW 2025*. Association for Computing Machinery, 422–431. doi:10.1145/3701716.3715225
[41] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *WWW 2025*. ACM, 777–780. doi:10.1145/3701716.3715290
[42] Prometheus. 2026. Open source metrics and monitoring for your systems and services. https://prometheus.io/.
[43] Pydantic. 2026. A widely used data validation library for Python. https://docs.pydantic.dev/latest/.
[44] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.
[45] Haotian Si, Jianhui Li, Changhua Pei, Hang Cui, Jingwen Yang, Yongqian Sun, Shenglin Zhang, Jingjing Li, Haiming Zhang, Jing Han, Dan Pei, and Gaogang Xie. 2024. TimeSeriesBench: An Industrial-Grade Benchmark for Time Series Anomaly Detection Models. In *ISSRE 2024*. IEEE, 61–72. doi:10.1109/ISSRE62328.2024.00017
[46] Stackoverflow. 2025. Stack Overflow. Retrieved June 10, 2026 from https://stackoverflow.com/questions
[47] Yongqian Sun, Jiaju Wang, Zhengdan Li, Xiaohui Nie, Minghua Ma, Shenglin Zhang, Yuhe Ji, Lu Zhang, Wen Long, Hengmao Chen, Yongnan Luo, and Dan Pei. 2025. AIOpsArena: Scenario-Oriented Evaluation and Leaderboard for AIOps Algorithms in Microservices. In *SANER 2025*. IEEE, 809–813. doi:10.1109/SANER64311.2025.00082
[48] Yifang Tian, Yaming Liu, Zichun Chong, Zihang Huang, and Hans-Arno Jacobsen. 2025. GALA: Can Graph-Augmented Large Language Model Agentic Workflows Elevate Root Cause Analysis? *CoRR* abs/2508.12472 (2025). arXiv:2508.12472 doi:10.48550/ARXIV.2508.12472
[49] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*. 2609–2634.
[50] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In *CIKM 2024*. ACM, 4966–4974. doi:10.1145/3627673.3680016
[51] Zirui Wang, Guangba Yu, and Michael R. Lyu. 2026. AI-NativeBench: An Open-Source White-Box Agentic Benchmark Suite for AI-Native Systems. *ACM Trans. Softw. Eng. Methodol.* (July 2026). doi:10.1145/3821576
[52] Zhilin Wang, Shaokun Zhang, Yifan Zhang, Hao Zhang, Jin Xu, Binfeng Xu, Jian Hu, Yunheng Zou, Karan Sapra, Andrew Tao, et al. 2026. OSWorld-Pro: Process-based Evaluation for Computer Use Agents. *arXiv preprint arXiv:2609.24890* (2026).
[53] Yong Xiang, Charley Peter Chen, Liyi Zeng, Wei Yin, Xin Liu, Hu Li, and Wei Xu. 2025. Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM. *CoRR* abs/2506.02490 (2025). arXiv:2506.02490 doi:10.48550/ARXIV.2506.02490
[54] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *ICLR 2025*. OpenReview.net. https://openreview.net/forum?id=M4qNIzQYpd
[55] Yitao Yang, Yangtao Deng, Yifan Xiong, Baochun Li, Hong Xu, and Peng Cheng. 2025. AidAI: Automated Incident Diagnosis for AI Workloads in the Cloud. *CoRR* abs/2506.01481 (2025). arXiv:2506.01481 doi:10.48550/ARXIV.2506.01481
[56] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
[57] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *WWW 2021*. ACM, 3087–3098. doi:10.1145/3442381.3449905
[58] Guangba Yu, Pengfei Chen, Zilong He, Qiuyu Yan, Yu Luo, Fangyuan Li, and Zibin Zheng. 2024. ChangeRCA: Finding Root Causes from Software Changes in Large Online Systems. *Proc. ACM Softw. Eng.* 1, FSE (2024), 24–46. doi:10.1145/3643728
[59] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *ESEC/FSE 2023*. ACM,

553–565. doi:10.1145/3611643.3616249

[60] Guangba Yu, Pengfei Chen, and Zibin Zheng. 2022. Microscaler: Cost-effective Scaling for Microservice Applications in the Cloud with an Online Learning Approach. *IEEE TCC* 10, 2 (2022), 1100–1116. doi:10.1109/TCC.2020.2985352

[61] Chaoli Zhang, Yingying Zhang, Lanshu Peng, Qingsong Wen, Yiyuan Yang, Chongjiong Fan, Minqi Jiang, Lunting Fan, and Liang Sun. 2024. Advancing Multivariate Time Series Anomaly Detection: A Comprehensive Benchmark with Real-World Data from Alibaba Cloud. In *CIKM 2024*. ACM, 5410–5414. doi:10.1145/3627673.3679128

[62] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Chiming Duan, Minghua He, Leyi Pan, Zhaoyang Liu, Bolin Ding, and Ying Li. 2025. MicroRemed: Benchmarking LLMs in Microservices Remediation. arXiv:2511.01166 [cs.CL] https://arxiv.org/abs/2511.01166

[63] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

[64] Renyi Zhong, Yichen Li, Guangba Yu, Wenwei Gu, Jinxi Kuang, Yintong Huo, and Michael R. Lyu. 2025. Larger Is Not Always Better: Exploring Small Open-source Language Models in Logging Statement Generation. *ACM Trans. Softw. Eng. Methodol.* (2025). doi:10.1145/3773287

[65] Yuedong Zhong, Guangba Yu, Yujie Huang, QunChao Fu, Rui Ren, Cong Feng, Yongqiang Yang, and Michael Lyu. 2026. LLMGuard: Multi-Agent Fault Diagnosis for Reliable Language-Model-as-a-Service. In *2026 56th Annual IEEE International Conference on Dependable Systems and Networks (DSN)*. 15–23. doi:10.1109/DSN69566.2026.00018

[66] Wei Zhou, Peng Sun, Xuanhe Zhou, Qianglei Zang, Ji Xu, Tieying Zhang, Guoliang Li, and Fan Wu. 2025. DBAIOps: A Reasoning LLM-Enhanced Database Operation and Maintenance System using Knowledge Graphs. arXiv:2508.01136 [cs.DB] https://arxiv.org/abs/2508.01136

[67] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Software Eng.* 47, 2 (2021), 243–260. doi:10.1109/TSE.2018.2887384

[68] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *ICSE 2018 Poster*. ACM, 323–324. doi:10.1145/3183440.3194991

[69] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *ISSRE 2023*. IEEE, 355–366. doi:10.1109/ISSRE59848.2023.00071